

Generic Queue<T> Collection Class in C# with Examples

In this article, I am going to discuss **Generic Queue<T> Collection Class in C#** with Examples. Please read our previous article where we discussed the [Generic Stack Collection Class in C#](#) with Examples. The Queue<T> is a Generic collection that stores elements in FIFO style (First In First Out). The C# Language Includes both the Generic Queue<T> and [Non-Generic Queue](#) collection classes. It is recommended by Microsoft to use the Generic Queue<T> Collection class as it is type-safe and boxing and unboxing it not required. Here in this article, we will discuss the Generic Queue Collection Class in C# with Examples. At the end of this article, you will understand the following pointers.

1. [What is Generic Queue<T> in C#?](#)
2. [How to Create a Generic Queue<T> Collection in C#?](#)
3. [How to Add Elements into a Queue<T> Collection in C#?](#)
4. [How to access a Generic Queue Collection in C#?](#)
5. [How to Remove Elements from a Generic Queue<T> Collection in C#?](#)
6. [How to Get the First Element from the Generic Queue in C#?](#)
7. [What is the difference between the Dequeue\(\) and Peek\(\) methods?](#)
8. [How do check whether an Element Exists or Not in the Generic Queue Collection in C#?](#)
9. [How to Copy a Generic Queue Collection to an Existing Array in C#?](#)
10. [Generic Queue Collection Class with Complex Types in C#](#)
11. [Generic Queue vs Non-Generic Queue in C#](#)

What is Generic Queue<T> in C#?

The Generic Queue in C# is a collection class that works on the principle of First In First Out (FIFO) and this class is present in System.Collections.Generic namespace. That means we need to go for Generic Queue Collection when we need First In First Out (FIFO) access to items.

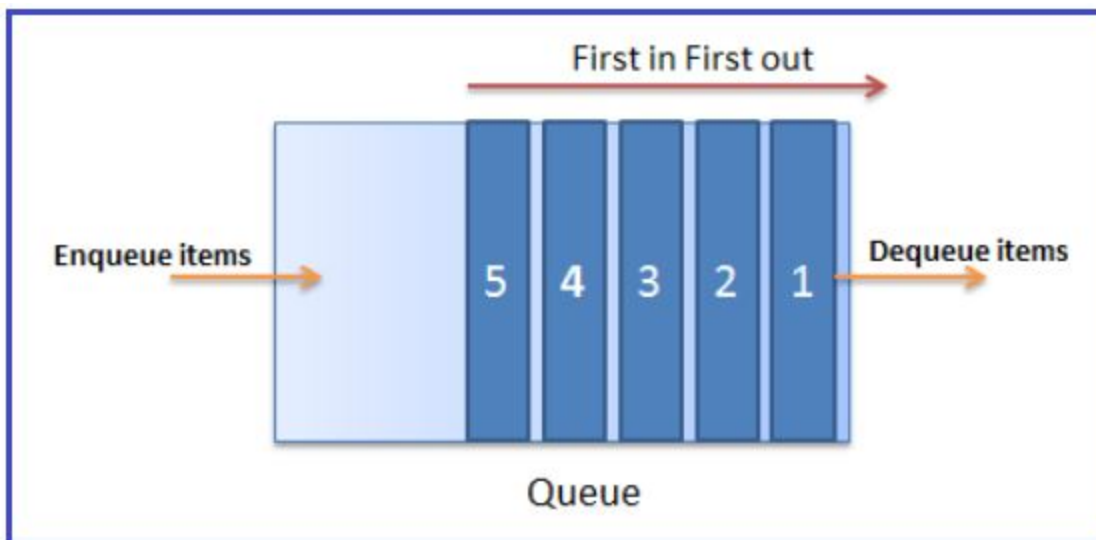
The Queue Collection Class is analogous to a queue at an ATM machine to withdraw money. The order, on which people queue up, will be the order in which they will be able to get out of the queue and withdraw money from the ATM. So, the First person who is in the queue will be the First person to withdraw the money and also the first person to be out of the ATM. For a better understanding, please have a look at the following image.

First Person in the Queue will server First



Queue - FIFO (First in First out)

The Queue collection class also operates in the same fashion. The first item to be added (enqueued) to the queue, will be the first item to be removed (dequeued) from the Queue. When we add an element into the queue, it is called en-queueing the element and when we remove an element from the queue, it is called dequeuing the element. For a better understanding, please have a look at the below image.



The capacity of a Queue is the number of elements the Queue can hold. As we added elements to a Queue, the capacity automatically increased. In Generic Queue Collection, we can store duplicate elements. A Queue can also accept null as a valid value for a reference type.

Methods, Properties, and Constructors of Generic Queue<T> Collection Class in C#:

If you go to the definition of Generic Queue<T> Collection class, then you will see the following. Here, you can see the Generic Queue class implements the IEnumerable<T>, IEnumerable, IReadOnlyCollection<T>, and ICollection interfaces.

```

namespace System.Collections.Generic
{
    ...public class Queue<T> : IEnumerable<T>, IEnumerable, IReadOnlyCollection<T>, ICollection
    {
        ...public Queue();
        ...public Queue(IEnumerable<T> collection);
        ...public Queue(int capacity);

        ...public int Count { get; }

        ...public void Clear();
        ...public bool Contains(T item);
        ...public void CopyTo(T[] array, int arrayIndex);
        ...public T Dequeue();
        ...public void Enqueue(T item);
        ...public Queue<T>.Enumerator GetEnumerator();
        ...public T Peek();
        ...public T[] ToArray();
        ...public void TrimExcess();
        ...public bool TryDequeue([MaybeNullWhen(false)] out T result);
        ...public bool TryPeek([MaybeNullWhen(false)] out T result);

        ...public struct Enumerator : IEnumerator<T>, IEnumerator, IDisposable
        {
            ...public T Current { get; }

            ...public void Dispose();
            ...public bool MoveNext();
        }
    }
}

```

How to Create a Generic Queue<T> Collection in C#?

The Generic Queue<T> Collection Class in C# provides the following three constructors to create an instance of the Generic Queue<T> class.

1. **Queue():** It is used to initialize a new instance of the Generic Queue class that is empty and has the default initial capacity.
2. **Queue(IEnumerable<T> collection):** It is used to initialize a new instance of the Generic Queue class that contains elements copied from the specified collection and has sufficient capacity to accommodate the number of elements copied. Here, the parameters collection specifies the collection whose elements are copied to the new Generic Queue. If the collection is null, then it will throw `ArgumentNullException`.
3. **Queue(int capacity):** It is used to initialize a new instance of the Generic Queue class that is empty and has the specified initial capacity. Here, the parameter capacity specifies the initial number of elements that the Queue can contain. If capacity is less than zero, then it will throw `ArgumentOutOfRangeException`.

Let's see how to create an instance of Generic Queue using the Queue() constructor:

Step1:

As the Generic Queue<T> class belongs to System.Collections.Generic namespace, so first, we need to

include System.Collections.Generic namespace into our program as follows:

using System.Collections.Generic;

Step2:

Next, we need to create an instance of the Generic Queue class using the Queue() constructor as follows:

Queue<type> Queue_Name = new Queue<type>();

Here, the type can be any built-in data type like int, double, string, etc., or any user-defined data type like Customer, Employee, Product, etc.

How to Add Elements into a Queue<T> Collection in C#?

If you want to add elements to a generic queue collection in C#, then you need to use the following Enqueue() method of the Queue<T> class.

1. **Enqueue(T item):** The Enqueue(T item) method is used to add an element at the end of the queue. Here, the parameter item specifies the element to add to the queue. The value can be null for a reference type i.e. when T is a reference type.

For example,

Queue<int> queue= new Queue<int>();

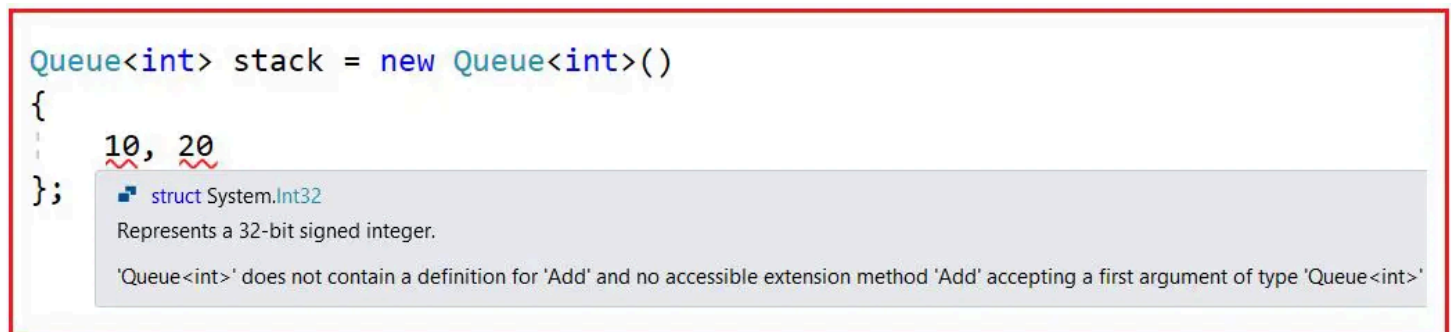
The above statement will create a generic queue of integer types. So, here we can only add integer-type elements on the queue. If we try to add anything other than an integer then we will get a compile-time error.

queue.Enqueue(10);

queue.Enqueue(20);

queue.Enqueue("Hello"); //Compile-Time Error

Note: We cannot add elements into a Queue Collection using Collection Initializer and the reason for this is the Generic Queue<T> Collection Class does not have an Add method. So, the point that you need to remember is if any collection class does not have the Add method then you cannot use Collection Initializer to add elements. If you try then you will get the compile-time error as shown in the below image.



How to access a Generic Queue Collection in C#?

We can access all the elements of the Generic Queue collection in C# using a for each loop as follows.

foreach (var item in queue)

{

Console.WriteLine(item);

}

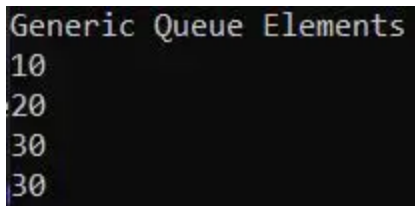
Note: You cannot use a for loop to access the elements of a generic queue collection and the reason for this is the Generic Queue collection class does not have any integer indexer.

Example to Understand How to Create a Generic Queue and Add Elements in C#:

For a better understanding of how to create a Generic Queue, how to add elements to a queue, and how to access all the elements from a queue in C# using a for-each loop, please have a look at the following example which shows the above three things.

```
using System;
using System.Collections.Generic;
namespace GenericQueueCollection
{
    public class Program
    {
        public static void Main()
        {
            //Creating a Queue to Store Integer Values
            Queue<int> queue = new Queue<int>();
            //Adding Elements to the Queue using Enqueue Method
            queue.Enqueue(10);
            queue.Enqueue(20);
            queue.Enqueue(30);
            //Adding Duplicate
            queue.Enqueue(30);
            //As int is not a Reference type so null can not be accepted by the
            //queue.Enqueue(null); //Compile-Time Error
            //As the queue is integer type, so string values can not be accepted
            //queue.Enqueue("Hello"); //Compile-Time Error
            //Accessing all the Elements of the Queue using For Each Loop
            Console.WriteLine("Generic Queue Elements");
            foreach (var item in queue)
            {
                Console.WriteLine(item);
            }
            Console.ReadKey();
        }
    }
}
```

Output:



```
Generic Queue Elements
10
20
30
30
```

How to Remove Elements from a Generic Queue<T> Collection in C#?

In Queue, the Elements which is added first will be the element to be removed first. That means we are allowed to remove elements from the beginning of the queue. The Generic Queue Collection Class in C#

provides the following two methods to remove elements.

1. **Dequeue():** This method is used to remove and return the object at the beginning of the Generic Queue. It returns the Object (element) removed from the beginning of the Generic Queue. If the Queue is empty, then it will throw `InvalidOperationException`.
2. **Clear():** This method is used to remove all objects from the Generic Queue.

Let us see an example to understand the `Dequeue()` and `Clear()` methods of the `Generic Queue<T>` Collection Class in C#. Please have a look at the following example which shows the use of the `Dequeue` and `Clear` method.

```
using System;
using System.Collections.Generic;
namespace GenericQueueCollection
{
    public class Program
    {
        public static void Main()
        {
            //Creating a Queue to Store Integer Values
            Queue<int> queue = new Queue<int>();
            //Adding Elements to the Queue using Enqueue Method
            queue.Enqueue(10);
            queue.Enqueue(20);
            queue.Enqueue(30);
            queue.Enqueue(40);
            queue.Enqueue(50);
            //Accesing all the Elements of the Queue using For Each Loop
            Console.WriteLine($"Generic Queue Elements Count: {queue.Count}");
            foreach (var item in queue)
            {
                Console.WriteLine(item);
            }
            // Removing and Returning an Element from the Begining of the Stack
            Dequeue method
            Console.WriteLine($"Deleted Element: {queue.Dequeue()}");
            //Printing Elements After Removing the First Added Element
            Console.WriteLine($"All Queue Elements After Deletion: Count {queue.Count}");
            foreach (var element in queue)
            {
                Console.WriteLine($"{element} ");
            }

            //Removing All Elements from Queue using Clear Method
            queue.Clear();
            Console.WriteLine($"All Queue Elements Counts After Clear: Count {queue.Count}");
            Console.ReadKey();
        }
    }
}
```

```
}
```

Output:

```
Generic Queue Elements Count: 5
10
20
30
40
50

Deleted Element: 10

All Queue Elements After Deletion: Count 4
20
30
40
50

All Queue Elements Counts After Clear: Count 0
```

How to Get the First Element from the Generic Queue in C#?

The Generic Queue<T> Collection Class in C# provides the following two methods to get the first element of the queue collection

1. **Dequeue():** The Dequeue() method of the Queue class is used to Remove and return the object from the beginning of the Queue. That means it returns the object that is removed from the beginning of the Generic Queue. If the queue is empty, it will throw InvalidOperationException
2. **Peek():** The peek() method of the Queue class is used to return the object at the beginning of the Queue without removing it. That means it returns the object from the beginning of the Queue. If the queue is empty, then it will throw InvalidOperationException.

For a better understanding, please have a look at the below example which shows how to get the first element from the queue using the Dequeue() and Peek() methods of Queue<T> Class in C#.

```
using System;
using System.Collections.Generic;
namespace GenericQueueCollection
{
    public class Program
    {
        public static void Main()
        {
            //Creating a Queue to Store Integer Values
            Queue<int> queue = new Queue<int>();
            //Adding Elements to the Queue using Enqueue Method
            queue.Enqueue(10);
```

```

queue.Enqueue(20);
queue.Enqueue(30);
queue.Enqueue(40);
queue.Enqueue(50);
//Accessing all the Elements of the Queue using For Each Loop
Console.WriteLine($"All Generic Queue Elements Count: {queue.Count}");
foreach (var item in queue)
{
    Console.WriteLine(item);
}
// Removing and Returning the First Element from queue using Dequeue
Console.WriteLine($"Dequeue Element: {queue.Dequeue()}");
//Printing Elements After Removing the First Added Element
Console.WriteLine($"All Queue Elements After Dequeue: Count {queue.Count}");
foreach (var element in queue)
{
    Console.WriteLine($"{element} ");
}
// Returning an Element from the Queue using Peek method
Console.WriteLine($"Peek Element: {queue.Peek()}");
//Printing Elements After Peek the Last Added Element
Console.WriteLine($"All Queue Elements After Peek: Count {queue.Count}");
foreach (var element in queue)
{
    Console.WriteLine($"{element} ");
}
Console.ReadKey();
}
}
}

```

Output:

```
All Generic Queue Elements Count: 5
10
20
30
40
50

Dequeue Element: 10
All Queue Elements After Dequeue: Count 4
20
30
40
50

Peek Element: 20
All Queue Elements After Peek: Count 4
20
30
40
50
```

What is the difference between the Dequeue() and Peek() methods?

The Dequeue() method removes and returns the item at the beginning of the queue, whereas the Peek() method returns the item at the beginning of the queue, without removing it. So, if you want to remove and return the first element from the queue, then use the Dequeue method and if you only want to return the first element from the queue without removing it, then use the Peek method and this is the only difference between these two methods of the Generic Queue<T> Collection Class in C#.

How to check whether an Element Exists or Not in the Generic Queue<T> Collection in C#?

If you want to check whether an element exists or not in the Generic Queue<T> Collection, then you need to use the following Contains() method provided by the Generic Queue Class in C#. Even, you can also use this method to search for an element in the given stack.

1. **Contains(T item)**: The Contains(T item) method is used to determine whether an element exists in the generic queue or not. It returns true if the item is found in the generic queue; otherwise, false. Here, the parameter item specifies the element to locate in the queue. The value can be null for a reference type.

Let us understand Contains(T item) Method with an example. The following example shows how to use the Contains() method of the Generic Queue Collection class in C#.

```
using System;
using System.Collections.Generic;
namespace GenericQueueCollection
{
```

```

public class Program
{
    public static void Main()
    {
        //Creating a Queue to Store Integer Values
        Queue<int> queue = new Queue<int>();
        //Adding Elements to the Queue using Enqueue Method
        queue.Enqueue(10);
        queue.Enqueue(20);
        queue.Enqueue(30);
        queue.Enqueue(40);
        queue.Enqueue(50);
        //Accesing all the Elements of the Queue using For Each Loop
        Console.WriteLine($"All Generic Queue Elements");
        foreach (var item in queue)
        {
            Console.WriteLine(item);
        }
        Console.WriteLine($"\\nIs Value 40 Present in the Queue:
{queue.Contains(50)}");
        Console.WriteLine($"\\nIs Value 90 Present in the Queue:
{queue.Contains(90)}");
        Console.ReadKey();
    }
}

```

Output:

```

All Generic Queue Elements
10
20
30
40
50

Is Value 40 Present in the Queue: True
Is Value 90 Present in the Queue: False

```

How to Copy a Generic Queue Collection to an Existing Array in C#?

In order to copy a Generic Queue Collection to an Existing Array in C#, we need to use the following CopyTo method of the Generic Queue Collection Class.

1. **CopyTo(T[] array, int arrayIndex):** This method is used to copy the Generic Queue Collection Elements to an existing one-dimensional Array, starting at the specified array index. Here, the parameter array specifies the one-dimensional array that is the destination of the elements copied from the generic queue. The Array must have zero-based indexing. The arrayIndex parameter

specifies the zero-based index in the array at which copying begins. If the parameter array is null, then it will throw `ArgumentNullException`. If the parameter index is less than zero, then it will throw `ArgumentOutOfRangeException`. If the number of elements in the source Generic Queue is greater than the available space from `arrayIndex` to the end of the destination array, then it will throw `ArgumentException`.

This method works on one-dimensional arrays and does not change the state of the generic queue. The elements are ordered in the array in the same way as the order of the elements from the beginning of the queue to the end. Let us see an example for a better understanding of the `CopyTo(T[] array, int arrayIndex)` method of the `Generic Queue<T> Collection Class` in C#.

```
using System;
using System.Collections.Generic;
namespace GenericQueueCollection
{
    public class Program
    {
        public static void Main()
        {
            //Creating a Queue to Store Integer Values
            Queue<int> queue = new Queue<int>();
            //Adding Elements to the Queue using Enqueue Method
            queue.Enqueue(10);
            queue.Enqueue(20);
            queue.Enqueue(30);
            queue.Enqueue(40);
            queue.Enqueue(50);
            //Accesing all the Elements of the Queue using For Each Loop
            Console.WriteLine($"All Generic Queue Elements");
            foreach (var item in queue)
            {
                Console.WriteLine(item);
            }
            //Copying the queue to an object array
            int[] queueCopy = new int[5];
            queue.CopyTo(queueCopy, 0);
            Console.WriteLine("\nQueue Copy Array Elements:");
            foreach (var item in queueCopy)
            {
                Console.WriteLine(item);
            }
            Console.ReadKey();
        }
    }
}
```

Output:

```
All Generic Queue Elements
```

```
10  
20  
30  
40  
50
```

```
Queue Copy Array Elements:
```

```
10  
20  
30  
40  
50
```

Generic Queue Collection Class with Complex Types in C#.

As of now, we have used the Generic Queue<T> Collection class with Primitive Data Types such as int, double, etc. Now, let us see how to use the Generic Queue<T> Collection class with complex types such as Employee, Customer, Product, etc. For a better understanding, please have a look at the below example where we use the Generic Queue<T> Collection with the user-defined Employee and perform different kinds of Operations on the queue. The following code is self-explained, so please go through the comment lines.

```
using System;  
using System.Collections.Generic;  
namespace GenericQueueDemo  
{  
    public class Program  
    {  
        static void Main(string[] args)  
        {  
            //Create Employee object  
            Employee emp1 = new Employee()  
            {  
                ID = 101,  
                Name = "Pranaya",  
                Gender = "Male",  
                Salary = 20000  
            };  
            Employee emp2 = new Employee()  
            {  
                ID = 102,  
                Name = "Priyanka",  
                Gender = "Female",  
                Salary = 30000  
            };  
            Employee emp3 = new Employee()  
            {  
                ID = 103,  
                Name = "Anurag",
```

```

        Gender = "Male",
        Salary = 40000
    };
    Employee emp4 = new Employee()
    {
        ID = 104,
        Name = "Sambit",
        Gender = "Female",
        Salary = 40000
    };
    Employee emp5 = new Employee()
    {
        ID = 105,
        Name = "Preety",
        Gender = "Female",
        Salary = 50000
    };
    // Create a Generic Queue of Employees
    Queue<Employee> queueEmployees = new Queue<Employee>();
    // To add an item into the queue, use the Enqueue() method.
    // emp1 is added first, so this employee, will be the first to get
queue
    queueEmployees.Enqueue(emp1);
    // emp2 will be queued up next, so employee 2 will be second to get
the queue
    queueEmployees.Enqueue(emp2);
    // emp3 will be queued up next, so employee 3 will be third to get
queue
    queueEmployees.Enqueue(emp3);
    // emp4 will be queued up next, so employee 4 will be fourth to get
the queue
    queueEmployees.Enqueue(emp4);
    // emp5 will be queued up next, so employee 5 will be fifth to get
queue
    queueEmployees.Enqueue(emp5);
    // If you need to loop thru each items in the queue, then we can use
foreach loop
    // in the same way as we use it with other collection classes.
    // The foreach loop will only iterate thru the items in the queue,
not remove them.
    // Notice that the items from the queue are retrieved in FIFO (First
Out), order.
    // The First element added to the queue is the first one to be retrieved
    Console.WriteLine("Retrive Using Foreach Loop");
    foreach (Employee emp in queueEmployees)
    {
        Console.WriteLine(emp.ID + " - " + emp.Name + " - " + emp.Gender
+ emp.Salary);
        Console.WriteLine("Items left in the Queue = " + queueEmployees.Count);
    }
    Console.WriteLine("-----");

```

```

        // To retrieve an item from the queue, use the Dequeue() method.
        // Notice that the items are dequeued in the same order in which they
enqueued.
        // Dequeue() method removes and returns the item at the beginning
Queue.
        // Since emp1 object is the one that is enqueued onto the queue first,
object will be
        // first to be dequeued and returned from the queue by using Dequeue()
        Console.WriteLine("Retrieve Using Dequeue Method");
        Employee e1 = queueEmployees.Dequeue();
        Console.WriteLine(e1.ID + " - " + e1.Name + " - " + e1.Gender + "
e1.Salary);
        Console.WriteLine("Items left in the Queue = " + queueEmployees.Count);
        Employee e2 = queueEmployees.Dequeue();
        Console.WriteLine(e2.ID + " - " + e2.Name + " - " + e2.Gender + "
e2.Salary);
        Console.WriteLine("Items left in the Queue = " + queueEmployees.Count);
        Employee e3 = queueEmployees.Dequeue();
        Console.WriteLine(e3.ID + " - " + e3.Name + " - " + e3.Gender + "
e3.Salary);
        Console.WriteLine("Items left in the Queue = " + queueEmployees.Count);
        Employee e4 = queueEmployees.Dequeue();
        Console.WriteLine(e4.ID + " - " + e4.Name + " - " + e4.Gender + "
e4.Salary);
        Console.WriteLine("Items left in the Queue = " + queueEmployees.Count);
        Employee e5 = queueEmployees.Dequeue();
        Console.WriteLine(e5.ID + " - " + e5.Name + " - " + e5.Gender + "
e5.Salary);
        Console.WriteLine("Items left in the Queue = " + queueEmployees.Count);
        Console.WriteLine("-----");
        // Now there will be no items left in the queue.
        // So, let's Enqueue the five objects once again
        queueEmployees.Enqueue(emp1);
        queueEmployees.Enqueue(emp2);
        queueEmployees.Enqueue(emp3);
        queueEmployees.Enqueue(emp4);
        queueEmployees.Enqueue(emp5);
        // To retrieve an item that is present at the beginning of the queue
        // without removing it, then use the Peek() method.
        Console.WriteLine("Retrieve Using Peek Method");
        Employee e101 = queueEmployees.Peek();
        Console.WriteLine(e101.ID + " - " + e101.Name + " - " + e101.Gender + "
e101.Salary);
        Console.WriteLine("Items left in the Queue = " + queueEmployees.Count);
        Employee e103 = queueEmployees.Peek();
        Console.WriteLine(e103.ID + " - " + e103.Name + " - " + e103.Gender + "
e103.Salary);
        Console.WriteLine("Items left in the Queue = " + queueEmployees.Count);
        Console.WriteLine("-----");
        // To check if an item exists in the stack, use Contains() method.
        if (queueEmployees.Contains(emp3))

```

```
        {
            Console.WriteLine("Emp3 is in Queue");
        }
        else
        {
            Console.WriteLine("Emp3 is not in queue");
        }
        Console.ReadKey();
    }
}
public class Employee
{
    public int ID { get; set; }
    public string Name { get; set; }
    public string Gender { get; set; }
    public int Salary { get; set; }
}
}
```

Output:

```

Items left in the Queue = 5
102 - Priyanka - Female - 30000
Items left in the Queue = 5
103 - Anurag - Male - 40000
Items left in the Queue = 5
104 - Sambit - Female - 40000
Items left in the Queue = 5
105 - Preety - Female - 50000
Items left in the Queue = 5
-----
Retrive Using Dequeue Method
101 - Pranaya - Male - 20000
Items left in the Queue = 4
102 - Priyanka - Female - 30000
Items left in the Queue = 3
103 - Anurag - Male - 40000
Items left in the Queue = 2
104 - Sambit - Female - 40000
Items left in the Queue = 1
105 - Preety - Female - 50000
Items left in the Queue = 0
-----
Retrive Using Peek Method
101 - Pranaya - Male - 20000
Items left in the Queue = 5
101 - Pranaya - Male - 20000
Items left in the Queue = 5
-----
Emp3 is in Queue

```

Note: To add items to the end of the queue, use the **Enqueue()** method. Similarly, to remove an item that is present at the beginning of the queue, use **Dequeue()** method. A foreach loop will iterate thru all the items in the queue. The peek method will return the item from the beginning of the queue but will not remove that item from the queue. To check if an item exists in the queue or not, use the **Contains()** method.

Generic Queue vs Non-Generic Queue in C#

1. The Generic Queue<T> Collection Class is defined under System.Collections.Generic namespace where the Non-Generic Queue Collection Class is defined under System.Collections namespace.
2. The Generic Queue<T> Class in C# can only store the same type of elements whereas the Non-Generic Queue Class can store the same or different types of elements as it operates on the object data type.
3. In Generic Queue<T>, we need to define the type of elements that we want to store in the queue. On the other hand, in a Non-Generic Queue, there is no need to define the type of elements that we want to store in the queue because it operates on object data type i.e. we can store any type of data.
4. The Generic Queue<T> is type-safe whereas the Non-Generic Queue is not type-safe.

C# Generic Queue<T> Collection Class Summary:

The following are some important points that you need to remember while working with Generic Queue Collection Class in C#.

1. The Queue Collection is used to store a collection of the same type of elements in a FIFO (First in, First out) manner, i.e., the element added first will be removed first.
2. As Queue<T> is a Generic Collection, so it comes under System.Collection.Generic namespace.
3. The Generic Queue<T> Collection stores elements of the specified type. It provides compile-time type checking and doesn't perform boxing-unboxing because it is generic.
4. By using the Enqueue() method, we can add elements to a queue collection. Here, we cannot use collection-initializer syntax to add elements to a queue.
5. The Dequeue() method will remove and return the first element from the beginning of the queue.
6. The Peek() method will return the first inserted element of the queue without removing it from the collection.
7. Queue Collection is very useful to store temporary data in the First In First Out (FIFO) style, where you might want to delete an element after retrieving its value.

In the next article, I am going to discuss **Foreach Loop in C#** with Examples. Here, in this article, I try to explain **Generic Queue Collection Class in C#** with Examples. I hope this Generic Queue<T> Collection Class in C# with Examples article will help you with your need. I would like to have your feedback. Please post your feedback, question, or comments about this article.